Blossom: a sectorized bloom filter with SIMD support

M3nny (github.com/M3nny)

October 8, 2026

1 Abstract

Blossom is a header-only **sectorized bloom filter** (SBF) library with an AVX-512 backend, and a scalar fallback, which exploits the same identical geometry and hashing schemes across backends.

This type of layout is particularly well-suited for hardware acceleration since one key touches only one cache line instead of `k`, for this reason vectorization leads to a speedup of $\sim 9\times$ when the filter fits in L1 cache compared to the scalar backend, down to $\sim 2\times$ when it is located in the DRAM (considering the lookup operation).

Not only does it allow for hardware acceleration, but it also favors the scalar backend due to the locality of the data, in fact **blocking** keys costs $3\times$ less memory time than spreading them all over the bit array (like in a classical bloom filter), this comes to the disadvantage of a higher delivered error rate at equal memory compared to the CBF.

2 Introduction

Bloom filters represent a data structure for **approximate membership** allowing for **false positives**, but **never false negatives** using far less space than storing the objects to be checked against (i.e. classic set).

They work by allocating a bit array, then a key is hashed by `k` independent hash functions where each one of them computes a bit to set to 1 in the bit array. Lookup in the filter works by doing the same procedure and checking if all the positions of an hashed key are set to 1, this may lead to false positives due to hash collisions with previously inserted keys, but it will never return a false negative, that is: if given a key, its `k` assigned bits are not all set to 1 it is sure that the element is not part of the set.

They are widely used in databases, crawling and networking systems in order to avoid expensive lookups in the main source of truth. However, this time saving matters only on a miss, otherwise (due to false positives), the system usually needs to also check the source of truth, hence this data structure earns its place when misses are common enough for those savings to accumulate.

The problem with the CBF is that `k` independent hashes in a large enough bit array land in `k` different cache lines, which causes many cache misses during a lookup, this is the main bottleneck at a large scale.

It is possible to constrain how keys are distributed across the bit array by applying another hash to a key to decide in which local region to set its `k` bits to 1. This approach is called **blocking**, and in this project we're going to use a specialization of it in order to further exploit space locality.

At 1% target false positive rate (FPR) we're going to use ≈ 9.7 bits/key instead of using 64 bits/key to store the raw `uint64` keys, which

is a 6.6× reduction in occupied space (not considering data structure overhead).

3 Background

The original bloom filter [1] introduced the data structure itself with a bit array of `m` bits and `k` independent hash functions, and gave a closed-form formula to compute FPR and how to minimize it by sizing correctly the filter. The main bottleneck was that the key distribution over a large bit array led to `k` cache misses.

Putze et al [2] introduced the block bloom filter by constraining keys inside a small block of bits, so an operation touches one cache line regardless of `k`, the price for this is a loss in accuracy. We can consider a block as its own CBF, since the keys are distributed randomly across blocks, some blocks will have an higher FPR which will contribute disproportionately to the filter's overall FPR rate, which makes it worse than what an evenly distributed filter would achieve.

Industry accepted this tradeoff, as we can see production libraries such as Parquet [3] implementing BBF by fixing `k=8` and confining those bits to a 256-bit block composed by 8 32-bit words, and sizing the filter by inverting the classical formula for finding m as if it was non-blocked, thus accepting the overshoot rather than precisely modeling it.

A later analysis by J. Apple [4] claims that for a target FPR in the range of $[0.4\%, 19\%]$ blocking remains a favorable trade, the Parquet specification highlights the underlying difficulty in sizing correctly a BBF by saying that "there is not a simple closed-form calculation of this probability" [5].

Jünger et al. [6] generalizes blocking to **sectorization**, where each block is split into `s` machine words and the `k` bits of a key are distributed evenly across them (k/s per word), so that each word becomes independently testable, this construction requires $k \geq s$ and is most regular when `k` is a multiple of `s`. Their target is GPU architectures, the present work implements the smallest case where `k=s`, on AVX-512.

Modern and separate lines of work optimize for space rather than locality. Cuckoo filters [7] and, more recently xor filters [8] approach the information-theoretic bound better than any other bloom filter variant, the latter is static and built once, and neither admits a cheap union between filters, whereas a BBF computes a merge by bitwise OR and can therefore be built in parallel.

Given the context where this work operates, where the filter exceeds the cache size, a probe's cost is dominated by memory latency, hence locality is a property worth optimizing.

4 Theoretical analysis

4.1 Layout

The filter's layout used in this project can be summarized by table 1

Symbol	Meaning	Value
n	keys inserted	Dynamically chosen
m	total bits	Dynamically chosen
B	bits per block	512
S	bits per word	64
$s = B/S$	words per block	8
k	bits set per key	8
k/s	bits per word per key	1

Table 1: Layout values used in the project.

The reason behind the block size being 512 bits large is that it is equal to the size of one AVX-512 register. Since the sectorized construction requires $k \geq s$ (so that each word has at least

one bit) and is most regular when s divides k, this work uses the smallest case where $k = s = 8$. This setting also makes self-collisions within a single key impossible.

4.2 Classical baseline

From the CBF, the probability that a given bit is set to 0 in the bloom filter (BF) is defined as:

$$\mathbb{P}(BF[i] = 0) = \left(1 - \frac{1}{m}\right)^{kn} \approx e^{-kn/m}$$

From this, we can derive the probability of a bit being 1:

$$\mathbb{P}(BF[i] = 1) = 1 - \mathbb{P}(BF[i] = 0) \quad (1)$$
$$= 1 - e^{-kn/m} \quad (2)$$

An element which is not part of the set is a false positive iff all of its k hash positions happen to be 1, and since the k hash functions are independent and each bit is 1, we define the probability of a false positive being:

$$P(\text{FP}) = \left(1 - e^{-kn/m}\right)^k$$

Minimizing over k, we obtain that the optimal number of bits set per key k^* is:

$$k^* = \frac{m}{n} \ln 2$$

The optimal number of bits per key m/n is then defined as:

$$\frac{m}{n} = \frac{-\ln p}{(\ln 2)^2}$$

So at $p = 1\%$, one would get ≈ 9.5 bits per key and $k^* \approx 6.6$.

4.3 Sizing

The filter's sizing formula is the same implemented by Parquet, that is:

$$m = \frac{-kn}{\ln\left(1 - p^{1/k}\right)}$$

Which is derived from the CBF formula with the only difference that m is not solved with the optimal k^*, but instead a fixed $k = 8$ (same as ours) in order to improve locality, however it is important to notice that the requested rate is what a (unblocked) CBF would deliver with the same parameters.

With $k = 8$ fixed, the same equation gives $\approx$ 9.7 bits per key at a 1% FPR target, against the ≈ 9.5 an optimally-chosen k would need, and even if $k = 8$ is not the optimal choice it is needed in exchange for one bit per word and one block per AVX-512 register.

4.4 Blocking costs accuracy

We've discussed that a key is assigned to a block by a hash, which means that on average a block has $\lambda = \frac{B}{m/n}$ keys, which at a 1% target FPR and $B = 512$ becomes $\lambda = 512/9.7 \approx 53$ keys per block.

Since every key sets only one bit in each word (8 in total) of the block, a single position is missed by one key with probability $1 - 1/S$, and the chance that all λ keys miss that bit is $(1 - 1/S)^\lambda = (63/64)^{53}$, so the chance that at least one key hits that bit (setting it to 1) is $1 - (63/64)^{53} \approx 56.5\%$, this means that after inserting the average number of keys λ, 56.5% of all bits inside each 64-bit word will be set to 1 (we refer to this as word fill).

To get a false positive during a lookup, a key that was never inserted before has to randomly hash to bits that are already set to 1 in all 8 words of the block, and the chance that every

word in a block happen to have a 1 where the key hashes to is equal to $0.565^8 \approx 0.0104$.

But a block won't receive all the λ keys (since λ is the average keys per block), and that 8th power is what makes the deviation asymmetric, in fact the error rate that comes from overloaded blocks is much higher than the error saving from underloaded blocks, so when putting them together, the overall FPR is worse than the CBF baseline.

4.5 The empirical FPR estimator

Predicting the FPR with a formula fails due to overloaded blocks, hence we can measure it directly by looking at the filter considering that a query triggers a false positive when the key hashes to a specific block, and that block happens to have set bits at all the s word positions the key checks.

1. Given the word j inside block b (we'll call it b_j), the probability of matching b_j (the word returns `true`) during a lookup, given that a word is composed by $S = 64$ bits is $\frac{\text{popcount}(b_j)}{S}$
2. The probability of matching a block during a lookup is given by multiplying (since each word hash is independent) the match probabilities of all $s = 8$ words together within itself: $\prod_{j=0}^{s-1} \frac{\text{popcount}(b_j)}{S}$
3. By adding up the the FPR for every individual block and averaging it across the entire filter, we get the total FPR: $P(\text{FP}) = \frac{1}{\#\text{blocks}} \sum_b \prod_{j=0}^{s-1} \frac{\text{popcount}(b_j)}{S}$

This approach costs us one pass of $m/64$ popcounts, but gives us a reliable measurement of the current FPR rather than estimating it with a formula that might not be precise considering the difficulty in finding such formula for this type of filter where blocks might be overloaded.

4.6 The cost model

CBF generates `k` hash values that map to arbitrary locations across the entire bit array, thus potentially leading to a read in multiple cache lines when the filter is big enough.

BBF guaranteed locality by setting the bits for a key inside space local block, thus leading to only 1 cache miss per lookup, since the blocks are sized to fit a CPU cache line, in our case it will load all the 512 bits onto L1 cache in a single access.

SBF makes the s word tests independent, since the blocks of 512 bits are split into $s = 8$ independent words, such that one SIMD instruction replaces the loop (instead of checking word by word).

The caveat is that vectorization makes a great difference only when the data is already sitting in the cache, however if the CPU is waiting for the data to arrive from the DRAM, the difference between the vectorized approach against the scalar one diminishes, since the lookup becomes memory bounded.

Moreover it is possible to state that due to Out-of-Order execution, the CPU will issue the reads from memory in parallel anyway, this is the reason why blocking does not actually have a `k`× speedup compared to the CBF, especially when the CPU is waiting for data from the DRAM.

5 Implementation details

The bitarray is implemented using a `std::vector` whose inner types are aligned `Block`s of 512 bits using `alignas(64)` since SIMD instructions like `_mm512_load_si512` require memory addresses to start at a 64 byte boundary (otherwise causing segmentation fault).

The inputs to the `lookup` and `insert` methods are normally `std::spans` of `std::uint64_t` keys, otherwise the passed type is required to be an iterable whose inner type implements a `std::hash` able to convert the iterated value into a `std::uint64_t` key, which will be used by the library's method override to automatically convert the input into a suitable type.

Since the input keys are already `std::uint64_t`, they are not subject to a full hashing pipeline, instead they are only transformed using a `splitMix64` [9] finalizer to evenly spread them, then a block is chosen via Lemire reduction [10] which avoids using the modulo operation. The bit inside each word of a block is selected with multiplicative hashing thus calling `splitmix64` once per key and then computing the bit position inside a word by multiplying the original hash by a precomputed salt.

The scalar backend iteratively probes 8 words using a standard loop, and early-exits as soon a key bit is not present in a word where it should have been present. The SIMD backend processes all 8 words in parallel using AVX-512 instructions.

Being an header-only library, the backend is dispatched at runtime by verifying the hardware capabilities, and throwing an exception if the user selected a backend not compatible with their computer architecture.

The output of the SIMD backend has been tested against the scalar one in order to ensure they output the same results for a given method.

Memory checks are enforced using AddressSanitizer and LeakSanitizer instead of Valgrind since it is not able to simulate AVX-512 instructions, which would leave the SIMD path untested (from the point of view of memory safety).

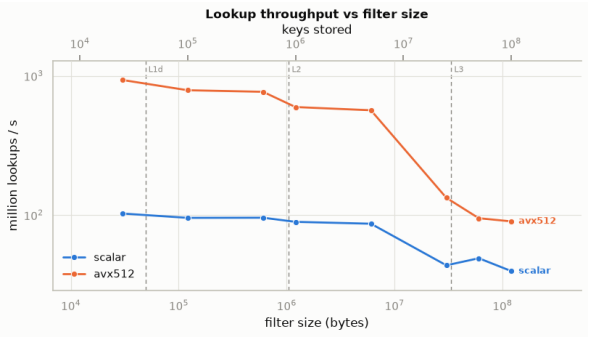


Figure 1: Lookup throughput vs filter size

6 Experimental results

The results shown in this section are produced using the following hardware:

- CPU: Ryzen 5 9600X, 48 KiB L1d + 1 MiB L2 per core, 32 MiB shared L3
- RAM: 32GB DDR5, 4800 MT/s

Every time measurement is taken by using the lowest time across 5 repetitions due to timings being very low in general and thus subject to OS noise.

The keys used to fill the filter are randomly generated using `std::mt19937_64`, except for the FPR benchmark discussed in subsection 6.4 which uses the `fb_200M_uint64` dataset [11] from facebook.

6.1 Throughput vs filter size

By testing the `lookup` throughput across various filter sizes, it is possible to see in figure 1 how locality helps both backends since when the filter fits the L1 cache the number of operations per second is higher compared to larger filters that spill out to the DRAM.

This difference is particularly visible for the SIMD backend, thus highlighting the memory-bound nature for this backend, and also showing the compute-bound behavior of the scalar

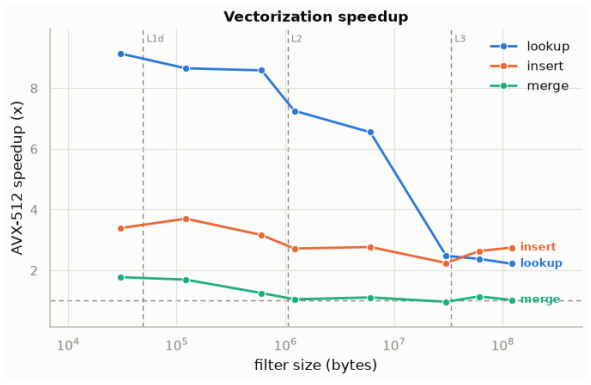


Figure 2: Vectorization speedup

backend.

Vectorizing improves the speed of the filter, but as it grows, its compute-bound nature becomes a memory-bound one, memory becomes a bottleneck and we start seeing diminishing returns.

6.2 Vectorization speedup

Figure 2 shows the speedup of the SIMD backend across the three filter operations, specifically the speedup range for each operation starting from the L1 cache down to the DRAM are:

- **Insert**: 3.39× → 2.75×
- **Lookup**: 9.14× → 2.21×
- **Merge**: 1.77× → 1.02×

As discussed in subsection 6.1, the `lookup` speedup is reasonably higher when the filter fits in the cache.

The `insert` operation is memory-bound from the start, that's why the speedup from vectorization is less compared to the lookup.

The `merge` method being just a repeated OR operation across the filter, does not gain the same advantage (as other operations) from vectorization since it is already a memory-bound operation.

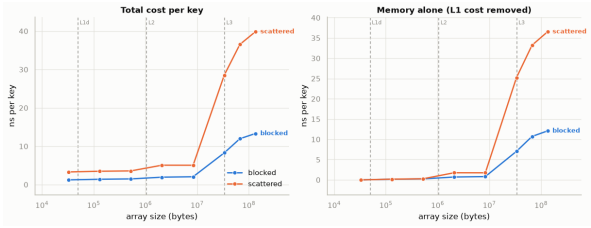


Figure 3: Keys locality effect

6.3 Locality

In order to show the effect of key locality (blocking), a benchmark has been conducted on a bit array with different sizes (not a bloom filter) by simulating two types of reads: the first one assuming that the bits of a key were allocated in a single block (the project's approach), and the second one by assuming that the bits of a key were scattered across the bit array (1 bit per block).

Figure 3 reports the results of this experiment, the gap in the left panel between the blocked and scattered approach at the beginning of the curve is due to indexing logic overhead (which reuses the methods from the bloom filter) since the scattered approach needs more multiplications since it has to select a distinct block per key bit, for this reason the right panel removes this time gap, since when the bit array fits in L1 cache every operation accesses only L1, hence it is not possible to deduce the potential advantage of blocking when considering small bit arrays.

The difference between these two approaches is clear when the array does not fit inside of L3, and at the largest size the scattered approach costs $\approx 3\times$ the memory time of the blocked approach.

This confirms what has been discussed in sub-

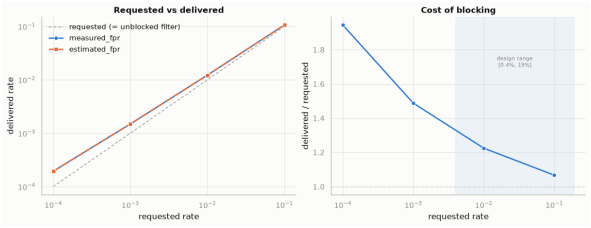


Figure 4: Requested vs delivered FPR

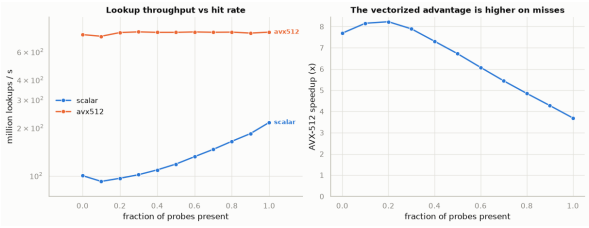


Figure 5: Lookup throughput vs hit rate

section 4.6 that is: the load operations are executed in parallel, so the speedup resulting from locality alone will never be $k\times$, but always below it.

6.4 False positive rate and the cost of blocking

This benchmark will measure how closely the filter respects the desired FPR requested by a user, for this reason instead of generating random keys (like in the other benchmarks) we are going to use the `fb_200M_uint64` dataset [11] which contains 200 million 64-bit unsigned integer keys, that should represent a real key distribution of IDs that are present in a database.

Only half of the keys are inserted into the filter (the backend does not matter in this experiment), the other half are used as probes, we define `to_insert` the keys to be inserted and `probes` the keys used as probes.

A false positive happens when a lookup of a probe returns "present", the measured FPR is then given by $\frac{\text{false positives}}{100{,}000{,}000}$.

Figure 4 shows in the left panel:

- **Requested**: the request FPR rate at filter build time ($y = x$)
- **Measured FPR**: the true FPR that has been measured by looking up the `probes`
- **Estimated FPR**: `estimated_fpr()` as discussed in subsection 4.5

The overshoot increases as FPR targets become smaller, this is due to the fact that we fixed `k` in order to exploit hardware acceleration, so the only parameter left that can be used to hit a target FPR is the average keys per block λ since overloaded blocks increase the FPR following what has been said in subsection 4.4. Lowering λ means allocating more blocks, and this over allocation is what makes 1-cache miss reads possible, but also makes the absolute FPR worse than the one of the CBF.

On the right panel it is reported the ratio between measured (delivered) and requested FPR, confirming what was claimed by section 3, that is: for a target FPR in the range of $[0.4\%, 19\%]$ blocking stays acceptable ($1.07\times$ at 10% and $1.22\times$ at 1%) compared to a CBF when talking about FPR accuracy, whilst the two targets that fall outside of that range show a significant deterioration ($1.49\times$ at 0.1% and $1.94\times$ at 0.01%).

6.5 Hit rate

This benchmark measures how many lookups per second are made based on data that is being queried. After allocating a filter that fits L2 cache (based on the number of expected elements), we perform various lookups against this filter by adding to the probes an incremental percentage of keys that are actually present inside of the filter, in this way we can measure how many lookups are completed in a second based on the percentage of probes that are ac-

tually present inside the filter.

Figure 5 shows that the number of lookups per second using the SIMD backend stays the same when increasing the fraction of probes present in the filter, however the scalar backend increases the throughput as more probes are present.

Initially this finding seemed misleading, but this is due to how the scalar backend is implemented. During a `lookup` the scalar backend early-exits as soon as it sees that a bit is not set on a word, causing the branch predictor to not understand a clear pattern when not fully completing the loop, for this reason, when 100% of the probed keys are actually present in the filter, the `lookup` method never early-exits, and it actually completes more lookups in a second.

Given this, the SIMD backend actually has a smaller speedup w.r.t. the scalar backend, only based on the data that is being queried, hence it would be possible to implement a branchless scalar lookup that would likely be faster than the current baseline.

7 Conclusions

Given the theoretical background and the experimental results, it is possible to state that at a 1% target FPR blocking costs 1.22× in actually delivered error rate, but buys 3.02× in memory time.

The vectorized approach pays off mostly when the filter fits inside the cache since it transforms a compute-bound operation into a memory-bound one.

The reported speedups are compared against a scalar baseline that implements early-exit, by using a branchless version the speedups would have been lower.

This filter architecture could be also implemented in modern GPUs in order to exploit their memory hierarchy and compute paradigm, which likely performs better for larger filters.

Bibliography

[1] Burton H. Bloom. "Space/time trade-offs in hash coding with allowable errors". In: *Commun. ACM* 13.7 (July 1970), pp. 422–426. ISSN: 0001-0782. DOI: 10.1145/362686.362692. URL: https://doi.org/10.1145/362686.362692.

[2] Felix Putze, Peter Sanders, and Johannes Singler. "Cache-, hash-, and space-efficient bloom filters". In: *ACM J. Exp. Algorithmics* 14 (Jan. 2010). ISSN: 1084-6654. DOI: 10.1145/1498698.1594230. URL: https://doi.org/10.1145/1498698.1594230.

[3] Apache Parquet. *BlockSplitBloomFilter.java*. commit 1687419. Apache Software Foundation. URL: https://github.com/apache/parquet-java/blob/master/parquet-column/src/main/java/org/apache/parquet/column/values/bloomfilter/BlockSplitBloomFilter.java (visited on 08/13/2026).

[4] Jim Apple. *Split block Bloom filters*. 2023. arXiv: 2101.01719 [cs.DS]. URL: https://arxiv.org/abs/2101.01719.

[5] Apache Parquet. *Bloom Filter Specification*. Apache Software Foundation. URL: https://github.com/apache/parquet-format/blob/master/BloomFilter.md (visited on 08/13/2026).

[6] Daniel Jünger et al. *Optimizing Bloom Filters on Modern GPUs*. 2026. arXiv: 2512.15595 [cs.DC]. URL: https://arxiv.org/abs/2512.15595.

[7] Bin Fan et al. “Cuckoo Filter: Practically Better Than Bloom”. In: *Proceedings of the 10th ACM International on Conference on Emerging Networking Experiments and Technologies*. CoNEXT ’14. Sydney, Australia: Association for Computing Machinery, 2014, pp. 75–88. ISBN: 9781450332798. DOI: 10.1145/2674005.2674994. URL: https://doi.org/10.1145/2674005.2674994.

[8] Thomas Mueller Graf and Daniel Lemire. “Xor Filters: Faster and Smaller Than Bloom and Cuckoo Filters”. In: *ACM Journal of Experimental Algorithmics* 25 (Mar. 2020), pp. 1–16. ISSN: 1084-6654. DOI: 10.1145/3376122. URL: http://dx.doi.org/10.1145/3376122.

[9] Guy L. Steele, Doug Lea, and Christine H. Flood. “Fast Splittable Pseudorandom Number Generators”. In: *Proceedings of OOPSLA*. Reference implementation: https://prng.di.unimi.it/splitmix64.c. 2014. DOI: 10.1145/2660193.2660195.

[10] Daniel Lemire. *A Fast Alternative to the Modulo Reduction*. 2016. URL: https://lemire.me/blog/2016/06/27/a-fast-alternative-to-the-modulo-reduction/ (visited on 08/13/2026).

[11] Ryan Marcus et al. *SOSD: Benchmarking Learned Indexes*. 2020. URL: https://doi.org/10.7910/DVN/JGVF9A (visited on 08/13/2026).